

# Lab 2 Pre-Exam

---

## 准备工作：创建并切换到 lab2-pre-exam 分支

---

1. 基于已完成的 lab2 提交自动初始化 lab2-exam 分支
2. 在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab2-pre-exam
```

初始化的 lab2-pre-exam 分支基于课下完成的 lab2 分支，并且在 tests 目录下添加了 lab2\_cond\_remove 样例测试目录。

## 题目背景 & 题目描述

---

在操作系统运行过程中，我们常需要取消地址映射。比如进程终止时清空整个地址空间的映射，或者进行物理页框的换出（swap），抑或是更改某段虚拟内存的保护级别。在本题中，你需要实现一个函数 `page_conditional_remove`，取消满足一定条件的页表项到对应的物理页框的映射。

```
u_int page_conditional_remove(Pde *pgdir, u_int asid, u_int perm_mask, u_long begin_va, u_long end_va);
```

该函数各参数的意义，函数的具体功能描述如下：

通过调用此函数，取消页目录 `pgdir` 对应的 **所有有效二级页表项**（下称 **页表项**）中，满足下面两个条件的页表项与对应的物理页框之间的映射。

- 虚拟地址范围在 `[begin_va, end_va)` 之间（注意区间开闭），测试样例保证 `begin_va` 和 `end_va` 被 `PAGE_SIZE` 整除；
- 权限位和 `perm_mask` 指定的权限位交集非空。

最后返回取消映射的页表项个数。

参数 `asid` 用于 `tlb` 一致性的维护。

## 题目要求

---

1. 在 `include/pmap.h` 中添加 `u_int page_conditional_remove` 函数的声明：

```
u_int page_conditional_remove(Pde *pgdir, u_int asid, u_int perm_mask, u_long begin_va, u_long end_va);
```

2. 在 kern/pmap.c 中实现该函数。

## 提示

---

可以仿照 page\_remove 实现这个函数，具体来说：

1. 可以使用 page\_lookup 函数实现页表项的查找
2. 可以使用 page\_decref 函数实现映射的取消
3. 记得使用 tlb\_invalidate 无效化tlb

## 样例输出 & 本地测试

---

对于以下样例：

```
#include <mmu.h>
#include <pmap.h>
#include <print.h>

void test_cond_remove() {
    struct Page *p;
    assert(page_alloc(&p) == 0);
    Pde *pgdir = (Pde *)page2kva(p);
    cur_pgdir = pgdir;
    u_int va[4] = {UTEXT, UTEXT + PAGE_SIZE, UTEXT + 1024 * PAGE_SIZE, UTEXT + 1025 * PAGE_SIZE};
    u_int perm[4] = {PTE_V, PTE_V | PTE_D | PTE_G, PTE_V | PTE_G,
                    PTE_V | PTE_D};
    struct Page *pp;

    assert(page_alloc(&pp) == 0);
    assert(page_insert(pgdir, 0, pp, va[0], perm[0]) == 0);
    assert(page_insert(pgdir, 0, pp, va[1], perm[1]) == 0);
    assert(page_insert(pgdir, 0, pp, va[2], perm[2]) == 0);
    assert(page_insert(pgdir, 0, pp, va[3], perm[3]) == 0);

    int r = page_conditional_remove(pgdir, 0, PTE_D | PTE_G, 0, UTEXT + 1025 * PAGE_SIZE);
    assert(r == 2);
    assert(pp->pp_ref == 2);
    assert(page_lookup(pgdir, va[0], 0));
    assert(page_lookup(pgdir, va[3], 0));
    assert(page_lookup(pgdir, va[1], 0) == 0);
    assert(page_lookup(pgdir, va[2], 0) == 0);
    printk("test succeeded!\n");
}
```

其应当输出：

```
test succeeded!
```

你可以使用：

make test lab=2\_cond\_remove\_pre\_1 && make run 在本地测试上述样例（调试模式）

MOS\_PROFILE=release make test lab=2\_cond\_remove\_pre\_1 && make run 在本地测试上述样例（开启优化）

或者在 init/init.c 的 mips\_init 函数中自行编写测试代码并使用 make && make run 测试。

## 提交评测 & 评测标准

请在开发机中执行下列命令后，在课程网站上提交评测。

```
$ cd ~/学号/
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

测试点说明及分数分布如下：

| 测试点序号 | 评测说明                    | 分值   |
|-------|-------------------------|------|
| 1     | 与样例相同                   | 10 分 |
| 2     | 测试数据仅检查函数的返回值           | 20 分 |
| 3     | 测试数据仅检查函数的返回值和页表项内容     | 25 分 |
| 4     | 测试数据仅检查函数返回值和物理页框结构体的维护 | 25 分 |
| 5     | 综合评测                    | 20 分 |

测试用例补充说明：

- end\_va - begin\_va 的值在 10^18 以内。
- 测试的权限位仅包括硬件位 D G 和软件位 LIBRARY COW ，不包括 cache 相关位，且无需考虑权限的语义。
- 保证 pgdir 和 asid 一一对应。